

GUIDA DEL FORMATORE

Corso di Formazione Professionale

Claude Desktop e Claude for Code

CAPITOLO 4

Spec-Driven Development

Materiale Completo per l'Erogazione della Formazione

Versione 1.0 - Gennaio 2025

Informazioni sul Documento

Questo documento costituisce la guida completa del formatore per il Capitolo 4 del corso Claude Desktop e Claude for Code.

Titolo Capitolo:	Spec-Driven Development
Target Audience:	Professionisti con competenze da principiante a intermedio
Livello:	Intermedio-Avanzato
Bilanciamento:	35% Teoria / 65% Pratica
Durata Stimata:	6-8 ore (distribuito su più sessioni)
Modalità:	Formazione remota via Microsoft Teams

Panoramica del Capitolo

Il Capitolo 4 rappresenta uno dei pilastri metodologici del corso, introducendo lo Spec-Driven Development come approccio distintivo per lo sviluppo assistito da AI.

Obiettivi di Apprendimento

Al termine di questo capitolo, i partecipanti saranno in grado di:

- Comprendere i principi fondamentali dello Spec-Driven Development
- Distinguere quando applicare questo approccio versus altri metodi
- Scrivere specifiche funzionali efficaci con il giusto livello di dettaglio
- Applicare template appropriati per progetti di diversa complessità
- Gestire il ciclo completo: specifica -> implementazione -> review -> raffinamento

Nota per il Formatore

Ogni sezione include note specifiche per il formatore con suggerimenti pedagogici e gestione dei tempi.

Indice

Informazioni sul Documento.....	2
Panoramica del Capitolo.....	2
Obiettivi di Apprendimento	2
Indice.....	3
1. Materiali Pre-Sessione	4
1.2 Apertura della Sessione.....	4
2. Topic 4.1: Fondamenti Concettuali.....	4
2.1 Fondamenti Teorici	4
Definizione e Principi Fondamentali	4
I Cinque Principi Cardine	5
Il Ciclo di Sviluppo.....	6
Vantaggi dello Spec-Driven Development.....	6
Confronto con Altri Approcci.....	6
Quando Usare lo Spec-Driven Development	7
Quando Altri Approcci Sono Piu Adatti.....	7
2.2 Dimostrazioni Pratiche	8
Esempio 1: Sistema Gestione Task Personali (Complessita: Bassa)	8
Esempio 2: Analizzatore Report Excel (Complessità: Media)	10
2.3 Esercizi Pratici	11
Esercizio 1: Identificare Quando Usare Spec-Driven	11
Esercizio 2: Scrivere Specifica da Requisiti Vaghi	12
Best Practices.....	13
Scrittura della Specifica.....	13
4.2 - Scrittura di Specifiche Funzionali Efficaci.....	14
4.2.1 Fondamenti Teorici	14
Anatomia di una Specifica Efficace	14
Template Pratici.....	15
Template 1: Progetto Piccolo (Script/Utility)	15
Template 2: Progetto Medio (Applicazione/Sistema)	16
Risorse Supplementari	17
Checklist: Quando Usare Spec-Driven.....	17
Glossario Termini Chiave.....	17

1. Materiali Pre-Sessione

1.2 Apertura della Sessione

L'apertura efficace crea il contesto giusto per l'apprendimento.

Domanda provocatoria: Come chiedereste a Claude di costruire un'app per tracciare attività quotidiane? Scrivetelo nella chat.

2. Topic 4.1: Fondamenti Concettuali

Questo topic introduce i principi fondamentali dello Spec-Driven Development.

2.1 Fondamenti Teorici

Definizione e Principi Fondamentali

Lo Spec-Driven Development è una metodologia strutturata per lo sviluppo software assistito da AI basata sulla creazione di specifiche funzionali dettagliate prima dell'implementazione.

A differenza di approcci informali, lo spec-driven development introduce una fase di pianificazione esplicita dove si definisce con precisione:

- **Cosa deve fare il software** (funzionalità richieste)
- **Come deve comportarsi** (logica e flussi)
- **Quali sono i requisiti** (funzionali e non-funzionali)
- **Quali sono i criteri di successo** (come verificare che funzioni)

I Cinque Principi Cardine

Lo Spec-Driven Development si basa su cinque principi fondamentali:

PRINCIPIO	DESCRIZIONE
1. Separazione Cosa/Come	La specifica definisce COSA il software deve fare, lasciando a Claude COME implementarlo. Il developer mantiene controllo strategico delegando lavoro tattico.
2. Granularità Controllata	Lo sviluppo procede in incrementi gestibili. Scomposizione del progetto in componenti sviluppabili, testabili e raffinabili iterativamente.
3. Review Iterativa	Ogni componente viene revisionato prima di procedere. Cicli di feedback rapidi per correzioni immediate.
4. Controllo Umano	Decisioni critiche (architettura, design, trade-off) rimangono responsabilità del developer. Claude implementa secondo direttive.
5. Documentazione Intrinseca	La specifica diventa documentazione del progetto. Descrive cosa fa, perché, quali requisiti soddisfa.



Perché Funziona

Lo Spec-Driven Development sfrutta i punti di forza di Claude (implementazione rapida, comprensione requisiti, codice di qualità) mentre mantiene il developer come architetto. Risultato: software di qualità in tempi ridotti.

Il Ciclo di Sviluppo

Lo Spec-Driven Development segue un ciclo iterativo in quattro fasi:

1. **SPECIFICA:** Scrittura della specifica funzionale dettagliata
2. **IMPLEMENTAZIONE:** Claude implementa secondo la specifica
3. **REVIEW:** Developer esamina aderenza e qualità
4. **RAFFINAMENTO:** Basato su review, si raffina specifica o implementazione

Vantaggi dello Spec-Driven Development

L'adozione di questo approccio offre benefici significativi:

- **Prevedibilità dei Risultati:** Specifiche chiare producono implementazioni coerenti
- **Qualità del Codice:** Codice aderisce a requisiti espliciti, include gestione errori
- **Manutenibilità:** La specifica documenta il perché dietro l'implementazione
- **Tracciabilità:** Ogni decisione di design è documentata
- **Riduzione Errori:** Requisiti espliciti eliminano ambiguità

Confronto con Altri Approcci

ASPETTO	SPEC-DRIVEN	PROMPT AD-HOC	SVILUPPO TRADIZIONALE
Pianificazione	Strutturata	Minima	Dettagliata
Prevedibilità	Alta	Bassa	Alta
Velocità	Media-Alta	Alta (iniziale)	Bassa
Qualità	Consistente e alta	Variabile	Alta
Manutenibilità	Eccellente	Scarsa	Buona

Attenzione

Lo Spec-Driven Development richiede investimento iniziale maggiore in pianificazione. Tuttavia ripaga rapidamente in progetti oltre la complessità triviale.

Quando Usare lo Spec-Driven Development

Lo Spec-Driven Development è particolarmente appropriato per:

- **Progetti di Media-Alta Complessità:** Più di 2-3 componenti, logica non triviale, requisiti articolati
- **Requisiti Chiari:** Sai cosa vuoi ma non è banale come costruirlo
- **Qualità Professionale:** Codice per produzione, non prototipo usa-e-getta
- **Collaboration Team:** Altri dovranno comprendere, mantenere, estendere il codice

Quando Altri Approcci Sono Più Adatti

- **Esplorazioni Rapide:** Sperimentare un'idea velocemente senza commitment
- **Prototipi Usa-e-Getta:** Demo che verranno scartate
- **Modifiche Minori:** Piccoli fix o aggiunte semplici
- **Script One-Off:** Utility che verranno eseguite poche volte

Regola Pratica

Se il progetto richiede più di 2 ore di sviluppo e il codice verrà usato per più di una settimana, vale la pena di applicare lo Spec-Driven Development.

2.2 Dimostrazioni Pratiche

Questa sezione presenta esempi completi che illustrano lo Spec-Driven Development in azione.



Nota per il Formatore

Dimostrare almeno 2-3 esempi live durante la sessione. Gli altri possono essere esplorati dai partecipanti in autonomia.

Esempio 1: Sistema Gestione Task Personali (Complessità: Bassa)

Contesto: Un professionista vuole un'app web per tracciare attività con priorità e scadenze.

APPROCCIO SENZA SPEC (Prompt Ad-Hoc):

Crea un'app web per gestire le mie task con priorità.

Risultato: App basic che manca funzionalità essenziali. Multiple iterazioni necessarie.

APPROCCIO CON SPEC (Spec-Driven):

SPECIFICA FUNZIONALE - Sistema Gestione Task

OVERVIEW

Applicazione web single-page per gestione task personali con priorità, scadenze, persistenza locale.

REQUISITI FUNZIONALI

1. Creazione Task con: titolo (obbligatorio), descrizione (opzionale), priorità (Alta/Media/Bassa), scadenza (data opzionale)
2. Visualizzazione lista con indicatori visivi per priorità (colori: rosso=alta, giallo=media, verde=bassa)
3. Gestione: marcare completati, editing, cancellazione con conferma
4. Organizzazione: filtro per priorità e stato, sorting per scadenza o priorità
5. Persistenza: salvare automaticamente in localStorage

REQUISITI NON-FUNZIONALI

- Interfaccia responsive (desktop e mobile) • Design pulito e intuitivo •
- Compatibilità: browser moderni

VINCOLI TECNICI

• Single file HTML (inline CSS + JS) • No dipendenze esterne • localStorage per persistenza

CRITERI DI ACCETTAZIONE

✓ Posso creare task con tutti i campi ✓ Task persistono dopo refresh ✓ Filtri funzionano correttamente ✓ Priorità visivamente distinguibili ✓ Posso editare e cancellare task ✓ Interfaccia utilizzabile su smartphone

Processo:

1. Fornire specifica completa a Claude
2. Claude genera file HTML completo
3. Review contro criteri di accettazione
4. Raffinamenti minori se necessario

Cosa Ha Funzionato Bene

La specifica dettagliata ha eliminato ambiguità. Claude sapeva esattamente cosa costruire. I criteri di accettazione hanno fornito checklist chiara per verifica.

Esempio 2: Analizzatore Report Excel (Complessità: Media)

Contesto: Un manager riceve report di vendita settimanali in Excel da diverse filiali. Vuole automatizzare l'estrazione dei dati chiave e la generazione di un sommario esecutivo.

APPROCCIO SENZA SPEC (Prompt Ad-Hoc):

"Crea uno script Python che legga i miei file Excel di vendita e generi un report."

Risultato: Script generico che richiede multiple iterazioni per gestire correttamente le colonne, i formati delle date, i casi edge. Manca gestione errori robusta.

APPROCCIO CON SPEC (Spec-Driven):



SPECIFICA FUNZIONALE - Analizzatore Report Vendite

OVERVIEW

Script Python per processare report Excel di vendita multi-filiale ed estrarre metriche chiave.

REQUISITI FUNZIONALI

1. Lettura file Excel (.xlsx) da directory specificata
2. Estrazione metriche: totale vendite, vendite per prodotto, trend vs settimana precedente
3. Validazione dati: colonne richieste presenti, formati corretti, nessun valore nullo nelle colonne critiche
4. Generazione sommario: file Markdown con tabelle riassuntive e indicatori chiave
5. Report errori: log dettagliato per file con problemi

REQUISITI NON-FUNZIONALI

- Performance: processare fino a 50 file in < 30 secondi
- Robustezza: continuare elaborazione anche se alcuni file hanno errori
- Manutenibilità: codice modulare con funzioni chiare

VINCOLI TECNICI

- Python 3.8+
- Librerie: pandas, openpyxl
- Input: directory con file .xlsx
- Output: sommario.md nella stessa directory

CRITERI DI ACCETTAZIONE

- ✓ Processa correttamente file Excel con schema atteso
- ✓ Genera sommario Markdown formattato correttamente
- ✓ Gestisce gracefully file mancanti o malformati
- ✓ Log dettagliato per debugging
- ✓ Esecuzione completata in tempo ragionevole (< 30s per 50 file)

Processo:

1. Fornire specifica completa a Claude
2. Claude genera script Python modulare con gestione errori
3. Test con dati reali, inclusi casi edge (file vuoti, colonne mancanti)
4. Review codice per robustezza e performance
5. Raffinamenti basati su test

✓ Cosa Ha Funzionato Bene

La specifica ha fornito chiarezza su: validazione richiesta, gestione errori, requisiti di performance. Claude ha generato codice con try-except appropriati e logging dettagliato. I criteri di accettazione hanno guidato il testing sistematico.

💡 Lezioni Apprese

Specificare requisiti non-funzionali (performance, robustezza) è critico per codice production-ready. I criteri di accettazione devono includere casi edge. La specifica ha eliminato 3-4 cicli di iterazione che sarebbero stati necessari con approccio ad-hoc.

2.3 Esercizi Pratici

Tre esercizi progressivi per sviluppare competenze nello Spec-Driven Development.

Esercizio 1: Identificare Quando Usare Spec-Driven

Obiettivo:	Sviluppare giudizio su quando applicare spec-driven vs altri approcci
Durata:	20 minuti
Modalità:	Individuale + discussione gruppo

Istruzioni:

Per ciascuno degli 8 scenari sotto, decidere se spec-driven è appropriato, eccessivo, o insufficiente. Giustificare.

#	SCENARIO	VALUTAZIONE
1	Script Python per rinominare 100 file	[da completare]
2	Sistema multi-utente prenotazione sale	[da completare]
3	Prototipo UI per testare idea app mobile	[da completare]
4	Libreria Python da pubblicare su PyPI	[da completare]
5	One-liner bash per estrarre email da log	[da completare]
6	Sistema gestione ordini e-commerce	[da completare]
7	Aggiungere bottone a pagina web esistente	[da completare]

8	Tool interno per report settimanali da 3 fonti	[da completare]
---	--	-----------------

Criteri di Successo:

- Valutazioni corrette per almeno 6/8 scenari
- Giustificazioni che dimostrano comprensione dei criteri

Soluzioni Suggerite:

5. ECCESSIVO - Task troppo semplice
6. APPROPRIATO - Complessità media, multi-utente, integrazione
7. ECCESSIVO - Prototipo esplorativo, requisiti fluidi
8. APPROPRIATO - Codice pubblico, qualità professionale
9. ECCESSIVO - One-liner, no spec necessaria
10. APPROPRIATO - Alta complessità, integrazione critica
11. ECCESSIVO - Modifica minore
12. APPROPRIATO - Complessità media, automazione, multi-fonte

Esercizio 2: Scrivere Specifica da Requisiti Vaghi

Obiettivo: Trasformare requisiti informali in specifica strutturata spec-driven

Durata: 25 minuti

Modalità: Individuale o coppie

Istruzioni:

Ricevete questa richiesta da un collega:

**Richiesta del Collega**

"Ho bisogno di qualcosa per tenere traccia delle mie ore di lavoro sui vari progetti. Vorrei poter aggiungere entry con progetto, ore, descrizione. Sarebbe utile vedere il totale ore per progetto e magari esportare i dati. Niente di complicato, solo qualcosa di semplice che funzioni."

Vostro compito:

Scrivere una specifica funzionale strutturata che includa:

- OVERVIEW: Descrizione concisa del sistema
- REQUISITI FUNZIONALI: Cosa deve fare (lista numerata, dettagliata)
- REQUISITI NON-FUNZIONALI: Usabilità, performance, compatibilità
- VINCOLI TECNICI: Tecnologie, formato file, deployment
- CRITERI DI ACCETTAZIONE: Checklist verificabile

Best Practices

Linee guida operative per applicare efficacemente spec-driven development nei vostri progetti.



Nota Didattica

Tempo stimato: 10-15 minuti. Presentare come checklist pratica. Incoraggiare partecipanti a bookmark questa sezione per riferimento futuro. Considerare di stampare come quick reference card.

Scrittura della Specifica

DO - Buone Pratiche:

- Investire tempo upfront: 10-20% del tempo totale progetto nella spec
- Essere specifici sui CRITERI DI ACCETTAZIONE: testabili, misurabili, chiari
- Includere ESEMPLI concrete: input/output attesi, edge cases, error scenarios
- Specificare il COSA e il PERCHÉ, lasciare flessibilità sul COME (a meno che vincolato)
- Usare TEMPLATE standardizzati per consistency across progetti
- Richiedere GESTIONE ERRORI esplicita: cosa fare quando qualcosa fallisce
- Definire REQUISITI NON-FUNZIONALI: performance, security, usability, maintainability

DON'T - Errori da Evitare:

- NON essere troppo prescrittivi sul 'come': soffoca soluzioni creative di Claude
- NON assumere knowledge implicita: esplicitare assumptions e constraints
- NON scrivere specs ambigue: 'performante', 'user-friendly' senza definizioni
- NON omettere edge cases e error conditions
- NON creare specs monolitiche per progetti complessi: scomporre in componenti

4.2 - Scrittura di Specifiche Funzionali Efficaci

Questa sezione approfondisce l'arte e la scienza di scrivere specifiche di alta qualità che guidano Claude verso implementazioni eccellenti.



Nota per il Formatore

Topic 4.2 rappresenta il cuore pratico dello spec-driven development. Obiettivo: trasformare partecipanti da consumers a producers di specifiche efficaci. Tempo stimato totale: 3.5-4 ore. Bilanciamento: 30% teoria / 70% pratica.

4.2.1 Fondamenti Teorici

Una specifica funzionale efficace è il ponte tra intenzione umana e implementazione AI. La sua qualità determina direttamente la qualità dell'output di Claude.

Anatomia di una Specifica Efficace

Ogni specifica di qualità contiene sette componenti essenziali. Ciascuna serve uno scopo specifico nel guidare Claude.

COMPONENTE	SCOPO
1. OVERVIEW	Contestualizza il problema. Risponde: Perché questo software esiste? Cosa risolve?
2. REQUISITI FUNZIONALI	Definisce COSA deve fare. Lista completa di capabilities e behaviors richiesti.
3. REQUISITI NON-FUNZIONALI	Specifica COME BENE deve funzionare. Performance, usability, security, maintainability.
4. VINCOLI TECNICI	Delimita lo spazio delle soluzioni. Tecnologie richieste, limitazioni da rispettare.
5. CRITERI DI ACCETTAZIONE	Rende la specifica testabile. Checklist verificabile di cosa costituisce successo.
6. CASI D'USO ED ESEMPI	Concretizza requisiti astratti. Mostra input/output attesi in scenari reali.
7. GESTIONE ERRORI	Anticipa failures. Specifica come sistema deve comportarsi quando qualcosa va storto.



Perché Tutte e Sette?

Omettere anche una singola componente crea lacune che Claude deve colmare con assumptions. Assumptions sono fonte primaria di misalignment tra atteso e ottenuto. Completeness è chiave per specifications efficaci.

Template Pratici

Questa sezione fornisce template pronti all'uso per specifiche di diversa complessità. Adattare questi template ai vostri progetti specifici.

Template 1: Progetto Piccolo (Script/Utility)



TEMPLATE: Specifica Progetto Piccolo

SPECIFICA FUNZIONALE: [Nome Progetto]

OVERVIEW

[1-2 frasi: Cosa fa lo script/utility? Problema che risolve?]

REQUISITI FUNZIONALI

1. [Funzionalità principale]
2. [Input handling]
3. [Output generazione]
4. [Edge cases management]

REQUISITI NON-FUNZIONALI

- **Performance:** [tempo esecuzione accettabile]
- **Usability:** [CLI user-friendly? Parametri chiari?]

VINCOLI TECNICI

- **Linguaggio:** [Python 3.x / Bash / etc.]
- **Dependencies:** [librerie richieste]
- **Input format:** [file, stdin, parametri CLI]
- **Output format:** [stdout, file, formato specifico]

CRITERI DI ACCETTAZIONE

- ✓ [Criterio testabile 1]
- ✓ [Criterio testabile 2]
- ✓ [Criterio testabile 3]
- ✓ Gestisce errori comuni gracefully
- ✓ Help message chiaro disponibile

Template 2: Progetto Medio (Applicazione/Sistema)



TEMPLATE: Specifica Progetto Medio

SPECIFICA FUNZIONALE: [Nome Progetto]

OVERVIEW

[Descrizione sistema: scopo, utenti, contesto]

ARCHITETTURA

- Componenti: [lista componenti principali]
- Integrations: [sistemi esterni, API]
- Data flow: [breve descrizione]

REQUISITI FUNZIONALI

1. [Feature set principale]
2. [User interactions]
3. [Data management]
4. [Business logic]
5. [Reporting/Output]

REQUISITI NON-FUNZIONALI

- Performance: [throughput, latency, concorrenza]
- Scalability: [volume dati, utenti]
- Reliability: [uptime, fault tolerance]
- Security: [authentication, authorization, data protection]
- Maintainability: [codice modulare, testabile, documentato]

VINCOLI TECNICI

- Stack: [framework, database, deployment platform]
- API specifications: [REST/GraphQL, versioning]
- Data models: [schema principale]
- External dependencies: [librerie, servizi terzi]

ERROR HANDLING

- Input validation: [come gestire input invalidi]
- Network errors: [retry policy, fallbacks]
- Business logic errors: [come comunicare a utente]
- Logging: [cosa loggare, formato, destination]

CRITERI DI ACCETTAZIONE

- ✓ [Feature completa e funzionante]
- ✓ [Performance soddisfa requisiti]
- ✓ [Gestione errori robusta]
- ✓ [Test suite passa]
- ✓ [Documentazione completa]

Risorse Supplementari

Checklist: Quando Usare Spec-Driven



Quick Decision Checklist

Considera Spec-Driven SE almeno 3 di questi sono TRUE:

- ☐ Progetto richiederà >2 ore di sviluppo
- ☐ Codice sarà usato per >1 settimana
- ☐ Altri dovranno comprendere/mantenere il codice
- ☐ Requisiti sono chiari ma implementazione non-triviale
- ☐ Quality è critica (production code, dati critici, etc.)
- ☐ Progetto ha multiple componenti con interdipendenze
- ☐ Error handling robusto è requisito
- ☐ Performance/scalability sono concerns

SE 5+ sono TRUE: Spec-driven è **HIGHLY RECOMMENDED**

SE 3-4 sono TRUE: Spec-driven è **BENEFICIAL**

SE <3 sono TRUE: Prompt ad-hoc può essere sufficiente

Glossario Termini Chiave

TERMINE	DEFINIZIONE
Specifica Funzionale	Documento strutturato che descrive COSA deve fare un software, includendo requisiti, constraints, criteri di successo
Requisiti Funzionali	Comportamenti specifici che il sistema deve implementare (features, capabilities)
Requisiti Non-Funzionali	Qualità e caratteristiche del sistema: performance, usability, security, maintainability
Criteri di Accettazione	Condizioni testabili che devono essere soddisfatte per considerare l'implementazione completa
Granularità Controllata	Scomposizione del progetto in componenti gestibili, sviluppabili e testabili indipendentemente
Ciclo Spec-Driven	Processo iterativo: Specifica → Implementazione → Review → Raffinamento
Edge Case	Scenario raro o estremo che testa limiti del sistema (es: input vuoti, volumi enormi, network failures)